

# IKT215 Exercise 8:

## Function Approximation Classification (Part II)

March 19<sup>th</sup>, 2026

*Exercise sessions, while not mandatory, are an integral part of your learning experience and exam preparation. Some exam questions will be based on tasks covered during these sessions. Although you have the flexibility to complete exercises at home and ask questions later (preferably within one week), I strongly recommend attending the sessions in person for immediate support and guidance. This allows you to benefit from direct interaction and real-time problem-solving. I respond promptly to emails ([manuel.s.mathew@uia.no](mailto:manuel.s.mathew@uia.no)).*

### Question 1: Neural Networks

#### Task 1: Non-linearities

Implement and compare all the activation functions discussed in the lecture.

1. Generate input data points ranging from -10 to 10.
2. Plot each activation function using matplotlib.
3. Discuss the advantages and disadvantages of each function.

#### Task 2: Neuron Implementation

Implement a single neuron in Python. The neuron should take an input vector  $x$ , weight vector  $w$ , bias  $b$ , and activation function  $f(\cdot)$ .

$$y = f(w^T x + b)$$

1. Define inputs as a numpy array, e.g.  $x = [1.0, -2.0, 3.0]$ .
2. Initialize weights and bias, e.g.  $w = [0.5, -0.6, 0.2]$  and  $b = 0.1$ .
3. Compute the output using ReLU activation.
4. Verify the output manually.

#### Task 3: MLP for XOR Problem

Using the single neuron implementation from Task 2, build a simple Multi-Layer Perceptron (MLP) with one hidden layer containing two neurons to solve the XOR classification problem.

MLP Structure:

- Input Layer: 2 neurons (XOR inputs)
- Hidden Layer: 2 neurons (ReLU activation)
- Output Layer: 1 neuron (Sigmoid activation)

XOR Dataset:

$$\{([X_1, X_2], Y)\} = \{([0,0], 0), ([0,1], 1), ([1,0], 1), ([1,1], 0)\}$$

1. Initialize weights and biases randomly.
2. Implement forward propagation through hidden and output layers.
3. Adjust weights manually to see if the problem can be solved.

#### Task 4: Backpropagation

Consider a simple neural network with the following architecture and parameters:

- Architecture: 2 input neurons, 2 hidden neurons, and 1 output neuron (2-2-1).
- Activation Function: Sigmoid ( $\sigma$ )
- Loss Function: Mean Squared Error (MSE)
- Learning Rate ( $\eta$ ): 0.1
- Initial Weights: All weights initialized to 0.5
- Biases: Initially set to zero

Given the following input-output pair:

- Input: [0.6, 0.4]
- Target Output: 1

#### 1. Manual Calculation (Forward and Backward Pass)

Perform one complete forward and backward pass manually, clearly showing all intermediate steps:

- a. Forward Pass
  - Compute the hidden layer inputs and outputs.
  - Compute the output layer input and final output.
  - Calculate the loss (MSE).
- b. Backward Pass
  - Compute gradients at the output node.
  - Compute gradients at each hidden node.
  - Update all weights using gradient descent.

#### 2. Python Implementation

Implement the backward pass in Python using NumPy arrays. Your function should perform the following:

- Accept a network dictionary containing weights (W1, W2) and biases (b1, b2).
- Accept input data (X) and target output (y\_true).
- Perform forward propagation internally to obtain intermediate activations.
- Compute gradients for all weights and biases through backpropagation.
- Return a dictionary containing these gradients.

#### 3. Verification

Compare your manual calculations from part (a) with your computational results from part (b). Verify that both methods yield consistent results.

### Task 5: MLP for Iris Dataset (PyTorch)

Using PyTorch (Or TensorFlow), create a simple MLP for classifying the Iris dataset based on all features.

1. Load Iris dataset
2. Split dataset into training and testing sets.
3. Implement MLP using PyTorch's `nn.Module` class.
4. Experiment with different:
  - a. Activation function
  - b. Weight Initialization
  - c. Learning rates and learning rate schedulers
5. Train model and evaluate accuracy on test set.
6. Discuss how different hyperparameter affects performance.

### Task 6: Decision Trees Using CART algorithm

Implement decision tree classification using CART algorithm in Python on iris dataset.

1. Load iris dataset.
2. Split data into training and testing sets.
3. Use `sklearn.tree.DecisionTreeClassifier` to implement CART-based decision tree.
4. Visualize decision tree structure using `sklearn.tree.plot_tree`.
5. Evaluate performance metrics such as accuracy on test set.
6. Experiment with hyperparameters such as maximum depth and minimum samples per leaf.
7. Compare accuracy with MLP for Iris Dataset developed in Task 5.